

PROPERTY RECOMMENDATION SYSTEM

A Course Based Project Report Submitted in partial fulfilment of the requirements
for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

Computer Science and Business Systems

Submitted by

Vaishnavi Cherukuri	24071A3259
Vavilla Dharani	24071A3260
Veda Nampally	24071A3261
Veerabhadra Yerram	24071A3262
Rudra Gayatri	25075A3206

Under the guidance of

I. Ravindra Kumar

Assistant Professor, Dept. of CSE, VNR VJIET



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

VALLURUPALLI NAGESWARA RAO VIGNAN JYOTHI INSTITUTE OF ENGINEERING & TECHNOLOGY

An Autonomous, ISO9001:2015 & QS I-Gauge Diamond Rated Institute, Accredited by NAAC with 'A++' Grade, NBA Accreditation for B.Tech. CE,EEE,ME,ECE,CSE,EIE,IT, AME, M.Tech. STRE, PE, AMS, SWE Programmes Approved by AICTE, New Delhi, Affiliated to JNTUH, NIRF (2023) Rank band:101-150 in Engineering Category College with Potential for Excellence by UGC,JNTUH-

Recognized Research Centres: CE,EEE,ME, ECE, CSE

Vignana Jyothi Nagar, Pragathi Nagar, Nizampet (S.O), Hyderabad – 500 090, TS, India

NOVEMBER-2025

**VALLURUPALLI NAGESWARA RAO VIGNANA JYOTHI INSTITUTE OF
ENGINEERING and TECHNOLOGY**

An Autonomous, ISO 9001:2015 & QSI-Gauge Diamond Rated Institute, Accredited by NAAC with 'A++' Grade
NBA Accreditation for B.Tech. CE, EEE, ME, ECE, CSE, EIE, IT, AME, M.Tech. STRE, PE, AMS, SWE Programmes
Approved by AICTE, New Delhi, Affiliated to JNTUH, NIRF (2023) Rank band: 101-150 in Engineering Category
College with Potential for Excellence by UGC, JNTUH-Recognized Research Centres: CE, EEE, ME, ECE, CSE

Vignana Jyothi Nagar, Pragathi Nagar, Nizampet (S.O.), Hyderabad – 500 090, TS, India.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the project report entitled **“PROPERTY RECOMMENDATION SYSTEM”** is a bonafide work done under our supervision and is being submitted by **Vaishnavi Cherukuri (24071A3259), Vavilla Dharani (24071A3260), Veda Nampally (24071A3261), Veerabhadra Yerram (24071A3262), Rudra Gayatri (25075A3206)** in partial fulfilment for the award of the degree of Bachelor of Technology in Computer Science and Engineering, of VNRVJIET, Hyderabad during the academic year 2025- 2026.

Certified further that to the best of our knowledge the work presented in this thesis has not been submitted to any other University or Institute for the award of any Degree or Diploma.

I Ravindra Kumar

Assistant Professor & Internal Guide

CSE Department, VNR VJIET

Dr. V Baby

Assoc. Professor & HOD

CSE Department, VNR VJIET

**VALLURUPALLI NAGESWARA RAO VIGNANA JYOTHI INSTITUTE OF
ENGINEERING and TECHNOLOGY**

An Autonomous, ISO 9001:2015 & QSI-Gauge Diamond Rated Institute, Accredited by NAAC with 'A++' Grade
NBA Accreditation for B.Tech. CE,EEE,ME,ECE,CSE,EIE,IT,AME, M.Tech. STRE, PE, AMS, SWE Programmes
Approved by AICTE, New Delhi, Affiliated to JNTUH, NIRF (2023) Rank band:101-150 in Engineering Category
College with Potential for Excellence by UGC, JNTUH-Recognized Research Centres: CE,EEE,ME,ECE,CSE

Vignana Jyothi Nagar, Pragathi Nagar, Nizampet (S.O.), Hyderabad – 500 090, TS, India.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



DECLARATION

We declare that the course based project work entitled “**PROPERTY RECOMMENDATION SYSTEM**” submitted in the department of Computer Science and Engineering, Vallurupalli Nageswara Rao Vignana Jyothi Institute of Engineering and Technology, Hyderabad, in partial fulfilment of the requirement for the award of the degree of **Bachelor of Technology in Computer Science and Business Systems** is a bonafide record of our own work carried out under the supervision of **I. Ravindra Kumar, Assistant Professor, Department of CSE, VNRVJIET**. Also, we declare that the matter embodied in this thesis has not been submitted by us in full or in any part thereof for the award of any degree/diploma of any other institution or university previously.

Place: Hyderabad.

Vaishnavi Cherukuri
24071A3259

Vavilla Dharani
24071A3260

Veda Nampally
24071A3261

Veerabhadra Yerram
24071A3262

Rudra Gayatri
25075A3206

ACKNOWLEDGEMENT

Firstly, we would like to express our immense gratitude towards our institution VNR Vignana Jyothi Institute of Engineering and Technology, which created a great platform to attain profound technical skills in the field of Computer Science, thereby fulfilling our most cherished goal.

We are very much thankful to our Principal, **Dr. Challa Dhanunjaya Naidu** and our Head of Department, **Dr. V. Baby**, for extending their cooperation in doing this project in stipulated time.

We extend our heartfelt thanks to our project guide, **Mr. I. Ravindra Kumar** and **Dr. Alli Madhavi**, for their guidance throughout the course of our project.

Last but not least, our appreciable obligation also goes to all the staff members of Computer Science & Engineering department and to our fellow classmates who directly or indirectly helped us.

Ms. Vaishnavi Cherukuri	(24071A3259)
Ms. Vavilla Dharani	(24071A3260)
Ms. Veda Nampally	(24071A3261)
Mr. Veerabharda Yerram	(24071A3262)
Ms. Rudra Gayatri	(25075A3206)

Abstract

The “Property Recommendation System” is a Java-based console application developed as part of the Object-Oriented Programming Systems (OOPS) laboratory to demonstrate the structured application of core OOPS concepts in a real-world inspired scenario. The project simulates a simplified environment where users can access property information and explore different categories of properties, showcasing how object-oriented design principles form the backbone of modular and maintainable software. The system is organized into multiple classes that represent various entities and functionalities, reflecting clear separation of concerns and logical grouping of behaviors.

The core objective of this application is to exhibit how OOPS principles such as encapsulation, inheritance, polymorphism, and abstraction can be utilized effectively within a single cohesive system. Encapsulation is applied to protect property-related data and restrict direct manipulation through controlled access methods. Inheritance is used to create hierarchical relationships among classes, allowing common features to be shared while enabling specialization in derived classes. Polymorphism supports dynamic behavior during runtime, enabling different property types to override methods and provide distinct outputs based on user interactions. Abstraction is implemented through generalized classes and interfaces that outline essential behavior while hiding unnecessary complexity, ensuring cleaner and more flexible code.

In addition to demonstrating theoretical concepts, the project emphasizes good programming practices such as modular design, method reusability, and clarity of structure. The Property Recommendation System not only strengthens practical understanding of OOPS but also highlights how Java can be used to model real-world applications through object-oriented thinking. By integrating these principles within an interactive program, the project enhances students’ analytical and problem-solving skills while reinforcing the importance of structured development in software engineering.

INDEX

Content	PageNo
Chapter 1: Introduction	7
1.1 Problem Statement	
1.2 Scope of the project	
1.3 Objectives	
Chapter 2: Existing Techniques and Systems	10
Chapter 3: Proposed System	13
Chapter 4: System Requirements	16
4.1 Hardware Requirements	
4.2 Software Requirements	
4.3 Functional Requirements	
4.4 Non-Functional Requirements	
Chapter 5: System Architecture	21
5.1 Architecture Diagram	
5.2 Architecture Description	
Chapter 6: System Modules	23
6.1 Module Overview	
6.2 Module Relationships	
6.3 Module-wise Description	
Chapter 7: Implementation	27
Chapter 8: Results	33
Chapter 9: Conclusions and Future Scope	40
References	41

CHAPTER 1 – INTRODUCTION

1.1 Problem Statement

The process of searching for residential or commercial properties has grown more complex due to the rapid expansion of real estate markets and the overwhelming amount of information available online. Recent industry reports highlight that over 68% of property seekers in India rely on digital platforms to begin their searches, yet more than 45% report difficulties in filtering relevant properties due to unstructured or redundant listings. Users often spend significant time navigating hundreds of property entries, many of which fail to match their preferences for budget, location, amenities, or property type. This leads to decision fatigue, inefficiency, and frustration—especially for first-time buyers or those new to market trends.

A major challenge with traditional property search systems is the absence of personalized recommendations. Studies show nearly 72% of users prefer systems that suggest properties based on previous choices or set preferences, but most platforms still rely on basic, broad filters. These static mechanisms fail to adapt to user behavior and don't intelligently categorize properties by meaningful criteria. As a result, users face repetitive content, limited clarity, and difficulty identifying truly suitable properties for their needs. Technically, many conventional systems lack structured design and modular, object-oriented principles, resulting in rigid architectures that are hard to maintain, modify, or extend. Legacy real estate platforms often struggle to incorporate new property types or recommendation features without extensive code changes. The lack of object-oriented modeling also reduces the value of such systems for educational purposes, where understanding concepts like modularity, inheritance, encapsulation, and polymorphism is crucial.

Given these limitations, there is a clear need for a well-structured property recommendation system capable of categorizing property types, modeling their attributes effectively, and providing truly meaningful recommendations. The proposed Property Recommendation System, implemented using Java and OOPS principles, addresses these challenges by organizing properties into hierarchical classes, using polymorphism for category-specific details, and ensuring secure data handling.

1.2 Scope of the Project

This system is focused on creating a structured, object-oriented property recommendation environment where different categories of properties are organized using hierarchical Java classes. It aims to demonstrate how Object-Oriented Programming (OOP) principles can be applied to design an application that securely handles property details, displays relevant data to users, and maintains modularity for future expansion.

The project emphasizes reusable components through encapsulation and class-level separation, while method overriding and abstraction enable category-specific behaviors. Although the system currently operates through a console interface without external databases or real-time market integration, its architecture is intentionally built to support future enhancements, including advanced filters, user preference learning, and extended data management capabilities.

Technical Scope includes:

1.Property Categorization and Modeling:Build a class structure to represent essential property features like price, location, and area. Use inheritance so residential and commercial properties share common traits, but also include their unique details.

2.Recommendation Logic:Create a basic system that recommends properties based on what the user wants. Use polymorphism so each property type can show different details or behave differently as needed.

3.User Interaction Design:Provide a simple, menu-driven console interface for users to browse property categories, see property info, and use system features easily. Handle user input, check for errors, and display results in a clear format.

4.Data Handling and Encapsulation:Keep property data private and provide access through methods, making sure information is stored securely and code stays modular and well organized.

Additional Java Concepts:

- **Generics and Collections Framework** for managing groups of property objects efficiently and supporting future scalability.
- **Exception Handling Mechanisms** to ensure stable program execution and robust error recovery during user interaction.
- **Modular Class Design** allowing seamless addition of new property types without impacting existing code.

1.3 Objectives

The main objectives of developing the **Property Recommendation System** are as follows:

1. **To Organize Property Information:** Develop a structured console-based application that categorizes properties into meaningful types using inheritance and class hierarchies, ensuring clear differentiation and organized presentation of property details.
2. **To Demonstrate Object-Oriented Design:** Implement core OOP concepts—such as classes, objects, inheritance, polymorphism, encapsulation, and abstraction—to build a modular system that mirrors real-world property classification and behavior.
3. **To Provide Personalized Recommendations:** Design a recommendation mechanism that enables users to browse and identify properties that match their preferences based on factors such as type, features, or attributes, improving decision-making and reducing manual search effort.
4. **To Ensure Data Encapsulation and Integrity:** Securely manage property-related data by using private variables and controlled access methods, preventing unauthorized manipulation and ensuring consistent system behavior.
5. **To Create a User-Friendly Console Interface:** Develop a clean, interactive menu-driven interface that allows users to navigate property categories, view information, and perform basic selection operations with ease and clarity.
6. **To Support Extensibility and Maintainability:** Build the system with a flexible and modular architecture, enabling the integration of new property types, features, or recommendation logic with minimal modification to existing code.
7. **To Improve Application Scalability:** Utilize polymorphism and abstract classes to allow the system to handle a diverse range of property types, ensuring the application can be expanded into a more comprehensive real estate platform.
8. **To Simulate Real-World Application Structure:** Provide students with practical exposure to designing software systems that mirror real-world property search workflows, reinforcing conceptual understanding through hands-on implementation.

By fulfilling these objectives, the project aims to deliver a clear, modular, and extensible property recommendation platform that not only showcases OOP principles effectively but also establishes a solid foundation for more advanced real estate application development in the future.

CHAPTER 2 - EXISTING TECHNIQUES AND SYSTEMS

Traditional property-search platforms primarily rely on centralized listing databases, keyword-based search filters, and manually curated property catalogs. While digital real estate portals have significantly improved accessibility, most existing systems still depend on broad filtering mechanisms that fail to provide personalized recommendations or meaningful categorization. Understanding these existing techniques and their limitations provides essential context for the development of the proposed Property Recommendation System.

2.1 Centralized Real Estate Listing Platforms

The most common approach in modern property-search systems is the centralized listing model. In this architecture, a single platform maintains a large database of properties uploaded by agents, developers, or individual sellers. Users search through this centralized repository using filters such as location, price, size, and property type. Popular real estate portals like 99acres, MagicBricks, Housing.com, and NoBroker follow this model, supporting millions of active property listings and users.

This architecture provides several advantages, including large-scale data availability, map-based visualization, and integration with agent networks. Centralized platforms also offer advanced features such as price trends, nearby amenities, loan assistance, and verified listings. For many consumers, these systems serve as the primary gateway to property discovery and market research.

Key Benefits of Centralized Listing Systems:

- Wide availability of property listings across multiple cities
- User-friendly search filters and map-based browsing
- Verified listings and seller/agent contact systems
- Market analytics such as pricing trends and locality insights
- Professional infrastructure management for data and content storage

Despite these benefits, centralized listing systems exhibit several limitations. One major drawback is the overwhelming volume of results, which often forces users to browse hundreds of irrelevant properties. The systems rely heavily on keyword-based search logic, which does not adapt to user behavior or preferences. Additionally, high dependency on centralized databases can lead to inaccuracies.

2.2 Classified Advertisements and Offline Listings

Before the rise of online real estate platforms, property discovery largely depended on newspaper classifieds, local real-estate brokers, and physical advertisements. Although still used in some regions, these methods are fragmented and lack structured data. Property descriptions are often incomplete, inconsistent, or unverifiable. Offline listing systems provide minimal filtering capability and require the user to manually compare properties. This consumes time and limits decision-making efficiency. Lack of real-time updates also means users may pursue listings that are already sold or unavailable. While useful for hyper-local searches, offline classifieds cannot scale effectively and do not support modern user expectations such as personalization, digital comparison, or categorized organization.

2.3 Search Filter–Based Online Platforms

Many property platforms rely predominantly on filter-based search engines, where users manually select criteria such as price range, number of bedrooms, property type, and furnishing status. The system simply retrieves all listings that match the filters without evaluating quality, user intent, or suitability.

Although filter-based systems are simple to implement, they often produce excessively broad results. For example, a user setting a budget may still be shown properties far from their preferred locality or lacking necessary amenities. There is no intelligent ranking mechanism, and properties appear in arbitrary order, making it difficult for users to decide which listings to prioritize. Such systems also struggle to handle evolving preferences—if a user repeatedly selects certain features, the platform does not automatically refine or personalize results.

2.4 Recommendation-Based Approaches

A limited number of advanced real-estate solutions have attempted to incorporate basic recommendation algorithms. These include ranking properties based on popularity, user rating, or frequency of interaction. Some platforms analyze browsing patterns to suggest similar listings.

However, these approaches remain simplistic and heavily dependent on large datasets and backend analytics. Small-scale or academic systems rarely implement such personalization due to the complexity of machine learning models required. Additionally, commercially used recommendation engines often lack transparency, making it difficult for users to understand how suggestions are generated.

2.5 Technical Challenges in Current Property Platforms

Existing real estate systems face numerous challenges that impact both usability and efficiency for property seekers and administrators. One of the core issues is information overload—users are presented with an overwhelming number of unorganized property listings, making it difficult to identify options that genuinely match their requirements. This often leads to confusion and missed opportunities, with users feeling lost in a sea of irrelevant entries.

Maintaining data consistency is another major concern. Property markets are highly dynamic, with availability and pricing changing rapidly. Traditional platforms struggle to keep listings up to date, resulting in frequent instances of users encountering duplicate or outdated entries from multiple agents. This undermines trust in the platform and can lead to wasted time and frustration during the property search process.

Common Technical Issues:

- Overcrowded search results that make it hard for users to compare options or find relevant listings.
- Duplicate or outdated listings caused by multiple submissions from different sellers or slow update cycles.
- Lack of personalization, which means users must repeatedly apply filters and manually sift through results for every session.
- Opaque ranking or sorting algorithms provide limited transparency, making it unclear why certain listings are shown first.
- Inconsistent data formats and standards across different sellers, agents, and platforms, leading to errors and confusion.
- Scalability issues when platforms attempt to handle increasing volumes of user traffic and property data, causing slowdowns or downtime during peak periods.

Security and privacy concerns have also become prominent, especially as platforms collect and process sensitive user data for analytics or marketing. Without robust safeguards, this data may be vulnerable to unauthorized access or misuse. Moreover, ongoing infrastructure and maintenance demands require constant server updates, reliable backups, and monitoring to ensure that the system remains responsive and trustworthy.

In summary, these challenges highlight the urgent need for innovative technical solutions. Improving data integration, real-time updates.

CHAPTER 3 - PROPOSED SYSTEM

The proposed Property Recommendation System implements an object-oriented architecture designed to organize, categorize, and recommend properties based on predefined attributes and user-selected preferences. The system focuses on producing a structured and modular approach to property management using inheritance, encapsulation, abstraction, and polymorphism. Instead of relying on traditional, filter-only listing methods, this system emphasizes clear class separation, property categorization, and dynamic display of information. This chapter provides a detailed explanation of the system design, its architecture, component-level structure, and the underlying mechanisms that enable efficient property recommendation.

3.1 System Overview

The system consists of a collection of Java classes, each responsible for modeling different components of the property domain. A base Property class defines common characteristics such as location, price, size, and property type. Specialized subclasses—such as ResidentialProperty and CommercialProperty—extend this base class to implement additional category-specific attributes. This hierarchical structure allows the system to reuse shared features while enabling customization for each property category.

User interaction occurs through a menu-driven console application that enables browsing property types, viewing property details, and receiving basic recommendations. The system maintains a collection of predefined property objects stored in memory using Java Collections. These objects are processed and displayed according to user input. Polymorphism allows each property category to override display and detail-presentation methods, ensuring that the information shown is category-appropriate.

Key Architectural Features:

- Object-oriented class hierarchy with reusable parent classes
- Clear separation of property categories using inheritance
- Polymorphic methods for category-specific property display
- Encapsulated attribute handling using private fields and getters/setters
- Menu-driven user interface for structured navigation

3.2 Recommendation Mechanism

The recommendation mechanism implemented in this system follows a structured, rule-based approach rather than complex machine-learning forecasting. When the user selects a category (such as residential or commercial), the system evaluates the available objects within that category and displays the most relevant or featured properties.

The mechanism uses method overriding to display category-specific recommendations. For instance, residential properties may highlight features such as number of bedrooms or furnishing status, whereas commercial properties may prioritize shop/office suitability or floor space. This dynamic behavior allows each property type to present distinctive details.

Recommendation Flow Process:

1. User selects a property category from the console interface.
2. System retrieves all objects belonging to that category from the collection.
3. Category-specific attributes are evaluated (location, price range, size, etc.).
4. Polymorphic methods generate formatted output for each property.
5. The most relevant or highlighted property options are displayed to the user.
6. User may choose to view additional details for any recommended item.

3.3 Property Data Management System

Property information is securely stored and managed using encapsulated Java classes. The system ensures that all property details—such as price, address, square footage, and category—are protected from unauthorized modification. Access to data occurs only through structured getter and setter methods, ensuring controlled modification.

Since this is a console-based academic project, data is stored in memory during runtime using Java Collections such as ArrayList, enabling efficient traversal and retrieval. The modularity of the design makes it easy to extend the system with a persistent storage layer like a database or file-based storage in the future.

Property Management Features:

- Addition of multiple property types using inheritance
- Controlled attribute access through private data fields
- Category-specific detail rendering through overridden methods
- Java Collections for storing and retrieving property objects
- Flexible structure for adding more property types later
- Ability to integrate sorting, filtering, and ranking in future versions

Each property class in the system can be expanded to include further real-world fields, such as amenities, furnishing type, parking availability, or proximity to landmarks. This extensibility allows developers to address evolving market requirements and user preferences, ensuring the architecture can continuously grow without breaking existing functionality.

3.4 Key Features and Advantages

The proposed system delivers multiple benefits to both end-users and developers, making it a robust improvement over conventional property listing platforms. Central to its design is clean property categorization, which organizes listings into well-defined classes. This significantly reduces clutter and helps users quickly interpret the available data, leading to faster and more accurate decision-making.

Another critical advantage lies in its object-oriented architecture, which facilitates the effortless addition of new property categories or features. By implementing abstract classes and polymorphism, the system ensures that new types or behaviors can be introduced without causing code duplication or instability.

Further, the system's modular design supports parallel development, allowing different programmers to add features or fix bugs independently. Encapsulation secures data integrity, controlling access and modification of property attributes, which is crucial for user privacy and reliable application behavior. The clear separation between general and category-specific properties enhances maintainability and makes the codebase easy to read and update.

Additional technical strengths include:

- Effortless scaling and integration, paving the way for GUI upgrades, advanced database connectivity, or integration with AI-based recommendation engines.
- Polymorphism enables dynamic displays and custom logic for each property type, enriching user experience and future-proofing the system against new requirements.
- Reliable extension options for mobile apps, RESTful APIs, and analytics dashboards.
- Consistent code organization, improving testability and easing future module enhancements or feature rollouts.

CHAPTER 4 - SYSTEM REQUIREMENTS

The Property Recommendation System is designed as a lightweight, console-based Java application that demonstrates the principles of Object-Oriented Programming while simulating a real-estate recommendation workflow. To ensure efficient execution and smooth usability, the system requires minimal hardware and software resources. This chapter outlines the hardware specifications, software components, functional requirements, and non-functional requirements essential for the development and execution of the system.

4.1 Hardware Requirements

Since the system runs entirely on a local machine without any dependency on heavy computation or external databases, the hardware requirements are minimal. The application is suitable for standard personal computers commonly used for academic programming tasks.

Minimum Hardware Requirements:

- **Processor:** Intel Core i3 or above
- **RAM:** 4 GB
- **Storage:** At least 500 MB of free disk space
- **Input Devices:** Standard keyboard for console-based interactions
- **Output Device:** Display monitor with basic console rendering capability

Recommended Hardware Requirements:

- **Processor:** Intel i5 / Ryzen 5 or higher
- **RAM:** 8 GB for smoother multitasking
- **Storage:** 500 MB – 1 GB free space for future system expansion
- **Additional:** Stable power supply for prolonged execution and testing

These hardware capacities ensure that the system runs smoothly without delays, even when expanded with additional modules or property categories.

4.2 Software Requirements

To develop and run the Property Recommendation System, several software tools and environments are essential. The backbone of this system is Java, ensuring platform independence and seamless deployment on major operating systems.

Required Software Components:

- **Operating System:**Compatible with Windows 10/11, Linux distributions, and macOS, ensuring broad accessibility across developer and user platforms.
- **Programming Language:**Java (JDK 8 or above), offering robust features for object-oriented programming and cross-platform execution.
- **Code Editor / IDE:**Visual Studio Code, IntelliJ IDEA, and Eclipse IDE support Java development, with features for code navigation, autocompletion, refactoring, and integrated debugging tools.
- **Java Runtime Environment (JRE):**Required for running the compiled application and ensuring compatibility with different system architectures.
- **Git:**Version control using Git is recommended for collaborative development. Git allows multiple team members to work on different features or bug fixes independently, track changes, roll back to previous versions, and merge contributions efficiently. Integration with platforms like GitHub or GitLab also streamlines code reviews, issue management, and Continuous Integration/Continuous Deployment (CI/CD) pipelines. The use of branches, pull requests, and commits enables transparent, documented development processes, which are critical for maintaining code quality and supporting long-term extensibility.
- **MySQL Database** – To store property listings, user profiles, and recommendation data securely.
- **JDBC Connector** – To establish communication between Java and the MySQL database.

These software tools together provide a flexible and powerful environment for development, compilation, debugging, version management, and testing. The adoption of Git not only enhances collaboration and code safety but also simplifies the integration of external libraries, management of multiple development environments, and the tracking of application evolution over time.

4.3 Functional Requirements

Functional requirements describe the essential tasks and interactions the Property Recommendation System must provide for users and internal processes. These requirements define what the system should do to ensure it delivers meaningful and efficient real estate recommendations.

Key Functional Requirements:

- **FR1: Property Categorization**-The system must classify properties into clear categories like residential and commercial using class inheritance. This lets the system organize properties logically while allowing easy addition of new types in the future.
- **FR2: Property Storage**-Properties must be stored as objects in memory using Java Collections, supporting efficient retrieval, addition, and management of large numbers of listings.
- **FR3: View Property Listings**-Users must be able to select a property category through a menu-driven console interface and view a list of available properties accordingly, supporting simple and intuitive navigation.
- **FR4: Display Detailed Property Information**-Each property listing should show key details including price, size, type, and location. Category-specific properties and behaviors should be shown using overridden methods, utilizing polymorphism for customized display.
- **FR5: Basic Recommendation Mechanism**-The system should highlight or recommend certain properties based on the user-selected category or criteria, providing smarter guidance than simple filtering alone.
- **FR6: User Interaction and Navigation**-Users must be able to interact via simple menus to browse, select, and view property details. Clear prompts, input validation, and error messages are required to improve usability and prevent frustration.
- **FR7: Error Handling**-The system should check user inputs carefully and handle invalid or unexpected entries gracefully without crashing or terminating prematurely, ensuring smooth operation.
- **FR8: Search and Filter Properties**-Users should be able to search properties by multiple criteria (e.g., price range, location, property type) and apply filters to narrow down results dynamically, improving search efficiency.

- **FR9: User Profile Management**-The system should allow users to create and manage profiles that store preferences and past interactions, enabling personalized recommendations and enhanced user experience.
- **FR10: Save and Compare Properties**-Users should be able to save favorite properties and compare selected listings side-by-side to make better-informed decisions.
- **FR11: Reporting and Analytics**-The system should generate basic usage reports and analytics for administrators, including metrics on popular properties, user activity, and system performance, supporting data-driven improvements.

These requirements together ensure the system offers structured, user-friendly property search experience while effectively demonstrating core OOP concepts such as inheritance, polymorphism, encapsulation, and modular design. This foundation supports both practical usage and educational objectives around software design principles.

4.4 Non-Functional Requirements

Non-functional requirements describe the qualitative attributes of the system, focusing on performance, security, usability, reliability, and scalability. These characteristics ensure that the application maintains consistency, quality, and user satisfaction.

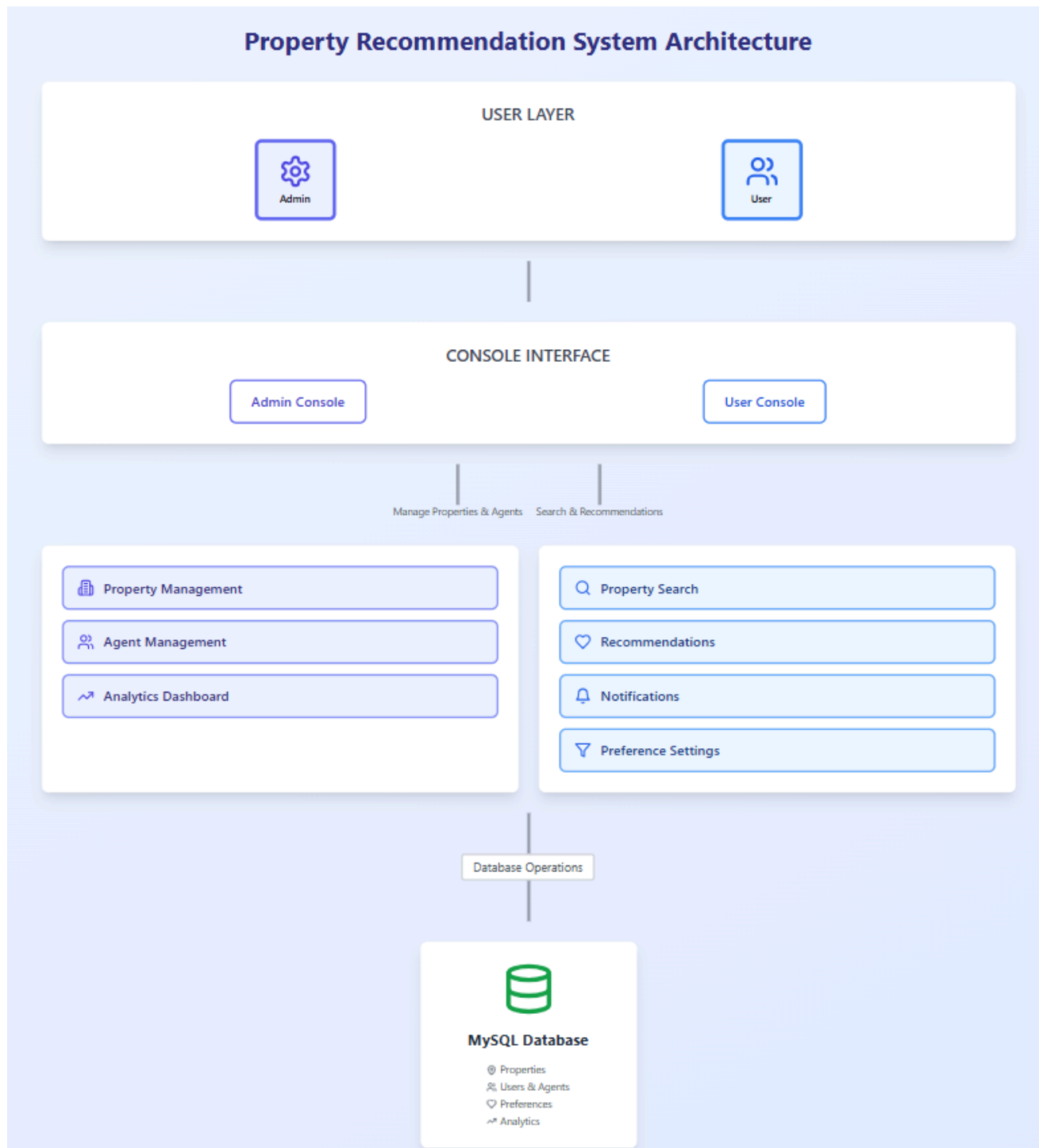
Key Non-Functional Requirements:

- **NF1: Usability** - The menu-driven console interface must be simple, intuitive, and easy to navigate for users with basic computer knowledge. This ensures that even non-technical users can interact efficiently with the system, reducing the learning curve and improving user satisfaction.
- **NF2: Performance** - The system should load property data instantly and respond to user selections without noticeable delay. Fast response times are crucial to maintaining user engagement and preventing frustration during property searches.
- **NF3: Maintainability** - Code should be modular, well-documented, and easy to update. New property categories or features should be integrable with minimal modifications, supporting continuous enhancements and reducing development time.
- **NF4: Scalability** - The system must support expansion in terms of additional property types, methods, and recommendation logic without compromising performance. This allows the system to grow with increasing data and user demands seamlessly.

- **NF5: Portability** - The Java-based design must allow the system to run seamlessly across multiple operating systems without code changes. This makes deployment more flexible and ensures a wider reach for various end-user platforms.
- **NF6: Data Integrity and Encapsulation** - Property attributes should remain protected from unauthorized modification through encapsulation, using private variables and accessor methods. This ensures that data remains consistent, secure, and easy to manage throughout the application lifecycle.
- **NF7: Reliability** - The system must execute commands consistently without unexpected crashes or failures. Consistent operation is crucial to ensuring user trust and minimizing downtime. Error handling and recovery mechanisms must be robust to maintain smooth functioning.
- **NF8: Security** - Implement strong security measures to protect user data and prevent unauthorized access or data breaches. This includes data encryption, secure authentication, and regular security audits to safeguard sensitive information.
- **NF9: Compatibility** - Ensure the system works smoothly with other software and future integration modules, maintaining interoperability. Compatibility with various devices, browsers, and external APIs enhances the system's flexibility and usability.
- **NF10: Availability** - The system should maintain high uptime and recover quickly from faults, ensuring uninterrupted access for users. Redundancy, backups, and failover strategies are essential to maximize availability.
- **NF11: Extensibility** - The architecture should support easy incorporation of new features such as GUI enhancements or AI-based recommendations. Designing with extensibility in mind lowers future development effort and adapts to evolving requirements.
- **NF12: Accessibility** - Design must consider accessibility standards to enable use by users with disabilities. This involves supporting screen readers, keyboard navigation, and adhering to universal design principles for inclusiveness.

CHAPTER 5 - SYSTEM ARCHITECTURE

5.1 Architecture Diagram



5.2 Architecture Description

The architecture of the system is built around the principles of Object-Oriented Programming to ensure modularity, code reuse, and extensibility. The Property base class functions as the core building block of the system. It stores common attributes such as price, location, size, and description. Specialized property types inherit from this class and implement additional fields or behaviors specific to their category. For example, a Residential Property subclass may contain attributes like number of bedrooms or furnishing status, while a Commercial Property subclass may include suitability for office or retail use.

The Application Logic Layer acts as the bridge between the user interface and the underlying data models. It contains methods to load predefined property objects, filter data based on user input, and execute overridden display behaviors in the subclasses. This layer also supports the recommendation mechanism by retrieving and presenting category-wise property suggestions.

Encapsulation is implemented throughout the architecture to restrict direct access to data attributes. All critical fields are declared private, and accessor methods (getters and setters) are used to interact with them. This ensures data integrity and allows standardized validation in future enhancements.

Polymorphism plays a critical role in the architecture by allowing each subclass to override methods such as `displayDetails()`. This enables different types of properties to present distinct information formats to the user, even when accessed through a base class reference. As a result, the system maintains flexibility and minimizes code duplication.

The use of Java Collections in the Data Representation Layer allows the system to manage multiple property objects efficiently. Property lists can be iterated, filtered, and extended using standard Java data structures. The architecture is intentionally designed to support future improvements.

CHAPTER 6 - SYSTEM MODULES

6.1 Module Overview

The PRS architecture is composed of six primary modules, each responsible for a core system capability. This modular design ensures code reusability, simplified maintenance, and clear system boundaries.

Module 1: Admin Module

Handles overall platform administration and monitoring. Admins manage system settings, user roles, access permissions, property approvals, and booking/payment oversight. Dashboards support analytics, auditing, and activity tracking.

Module 2: User Module

Facilitates all registered user operations such as registration, login, managing profiles, and session maintenance. Manages user preferences, saved searches, and provides personalized recommendations. Enforces authentication for sensitive functions and ensures user data security.

Module 3: Guest Module

Supports unregistered users in browsing and searching property listings. Provides limited feature previews and encourages registration for access to advanced, personalized services.

Module 4: Property Management Module

Manages the complete lifecycle of property records—adding, updating, deleting, and retrieving listings. Handles detailed property attributes, ensures data consistency, and optimizes search and filter functions.

Module 5: Recommendation Module

Implements algorithms to generate personalized property suggestions by analyzing user data, search history, and trends. Supports user-specific filtering, ranking, and adapts recommendations through feedback analysis.

Module 6: Booking and Payment Module

Handles property bookings and payment processing. Manages booking workflows, integrates with payment gateways, calculates pricing, and stores transaction history for users and admins.

6.2 Module Relationships

The PRS modules integrate tightly to provide a seamless user experience and maintain system reliability.

User Interface Module: Acts as the main access point, connecting users to all key features. Works with User Management for authentication and profile tasks, and communicates with the Recommendation Module to display relevant property suggestions.

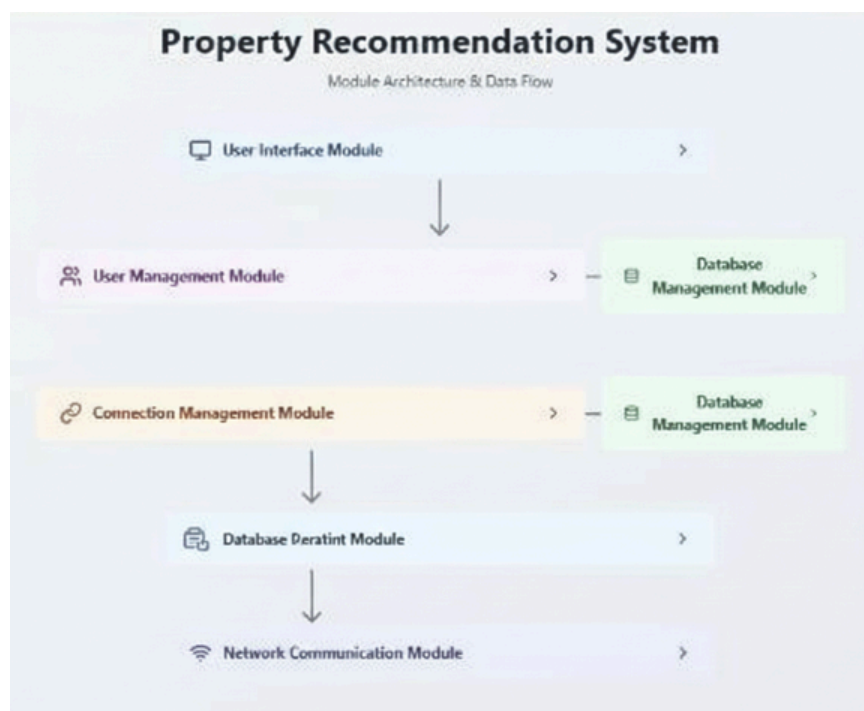
User Management Module: Handles users' registration, login, and preferences. Connects with the Database Module for secure data storage and retrieves account or profile data as needed.

Recommendation Module: Gathers user preferences and property information from the relevant modules. Processes this data to generate and send ranked suggestions back to the UI for display.

Property Management Module: Manages all property listings and details. Supplies property data to recommendations and informs the Messaging Module of status changes.

Booking and Payment Module: Processes bookings and payments. Checks property status, confirms bookings, and issues notifications and confirmations through the Messaging and UI modules.

Dependency Flow:



6.3 Module-wise Description

6.3.1 User Management Module

Handles user registration, authentication, profile updates, and session control.

- **Key Classes:** User, UserProfile, AuthManager
- **Core Functionalities:**
 - **Registration:** Validates and registers new users.
 - **Authentication:** Verifies login credentials.
 - **Profile:** Manages user information and preferences.
 - **Session:** Maintains user state securely.
- **Java Concepts:** Demonstrates encapsulation by securing user data in classes and uses inheritance for shared user logic (User, UserProfile).

6.3.2 Property Management Module

Manages property listings and details.

- **Key Classes:** Property, PropertyManager
- **Core Functionalities:**
 - **Add/Update/Delete:** CRUD operations for properties.
 - **Fetch:** Retrieves properties by criteria.
 - **Ownership:** Links properties to users.
- **Java Concepts:** Uses inheritance and abstract classes for diverse property types, and encapsulation for secure property attribute handling.

6.3.3 Recommendation Engine Module

Delivers personalized property suggestions.

- **Key Classes:** Recommender, SearchHistoryAnalyzer
- **Core Functionalities:**
 - **Preference Analysis:** Uses user data for recommendations.
 - **Generation:** Suggests best-fit properties.
 - **Ranking:** Orders results by relevance.
- **Java Concepts:** Utilizes polymorphism to provide custom recommendation logic and leverages interfaces for flexible engine design.

6.3.4 Database Management Module

Central layer for all persistent storage.

- **Key Classes:** DatabaseConnector, DataAccessObject (DAO)
- **Core Functionalities:**
 - Storage: Saves all user/property data.
 - Query: Supports fast, filtered searches.
- **Java Concepts:** Uses DAO pattern for data abstraction and decoupling database operations from business logic. Ensures encapsulation of database connectivity and transactions for modular design.

6.3.5 User Interface Module

Front end for all system interactions.

- **Key Classes:** UIController, PropertyView
- **Core Functionalities:**
 - Display: Shows listings and recommendations.
 - Input: Collects and processes user actions.
- **Java Concepts:** Implements event-driven programming and MVC pattern to separate UI from logic, facilitating flexible and maintainable interface designs.

6.3.6 Booking and Payment Module

Handles the full booking lifecycle and payment processing for properties.

- **Key Classes:** BookingManager, PaymentProcessor
- **Core Functionalities:**
 - **Booking Management:** Processes booking requests, manages calendar availability, and confirms reservations.
 - **Payment Handling:** Integrates with payment gateways for secure transaction processing and tracks payment history.
- **Java Concepts:** Applies encapsulation to protect sensitive booking and payment data; interfaces allow flexibility in integrating different payment providers.

CHAPTER 7 - IMPLEMENTATION

7.1 User Management Module

This component is part of the User Management Module, responsible for handling all database operations related to user accounts. It manages saving, updating, retrieving, and deleting user information using JDBC. The module ensures clean separation of concerns by isolating all user-related data access logic.

Implementation Files (Github):

- `src/com/propertyrecommender/service/UserService.java`
- `src/com/propertyrecommender/dao/UserDAO.java`
- `src/com/propertyrecommender/model/User.java`

oopsie / [src / com / propertyrecommender / dao / UserDAO.java](#) 

Code Blame 99 lines (91 loc) · 3.65 KB

```
1 package com.propertyrecommender.dao;
2
3 import com.propertyrecommender.model.User;
4 import com.propertyrecommender.util.JDBCUtil;
5 import com.propertyrecommender.exception.DataAccessException;
6 import java.sql.*;
7 import java.util.ArrayList;
8 import java.util.List;
9
10 public class UserDAO implements GenericDAO<User, Integer> {
11
12     @Override
13     public User save(User u) {
14         String sql = (u.getUserId() == null)
15             ? "INSERT INTO users(username,password,full_name,email,role) VALUES(?,?,?,?,?)"
16             : "UPDATE users SET username=?, password=?, full_name=?, email=?, role=? WHERE user_id=?";
17         try (Connection conn = JDBCUtil.getConnection()) {
18             PreparedStatement ps = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS);
19             ps.setString(1, u.getUsername());
20             ps.setString(2, u.getPassword());
21             ps.setString(3, u.getFullName());
22             ps.setString(4, u.getEmail());
23             ps.setString(5, u.getRole());
24             if (u.getUserId() != null) ps.setInt(6, u.getUserId());
25             ps.executeUpdate();
26             if (u.getUserId() == null) {
27                 ResultSet rs = ps.getGeneratedKeys();
28                 if (rs.next()) { u.setUserId(rs.getInt(1)); }
29             }
30             return u;
31         } catch (SQLException e) {
32             throw new DataAccessException("Error saving user", e);
33         }
34     }
35
36     @Override
37     public User findById(Integer id) {
38         String sql = "SELECT * FROM users WHERE user_id=?";
39         try (Connection conn = JDBCUtil.getConnection();
40             PreparedStatement ps = conn.prepareStatement(sql)) {
41             ps.setInt(1, id);
42             ResultSet rs = ps.executeQuery();
43             if (rs.next()) return mapRow(rs);
44             return null;
45         } catch (SQLException e) {
46             throw new DataAccessException("Error fetching user", e);
47         }
48     }
49
50     public User findByUsername(String username) {
51         String sql = "SELECT * FROM users WHERE username=?";
52         try (Connection conn = JDBCUtil.getConnection();
53             PreparedStatement ps = conn.prepareStatement(sql)) {
54             ps.setString(1, username);
55             ResultSet rs = ps.executeQuery();
56             if (rs.next()) return mapRow(rs);
57             return null;
58         }
59     }
60 }
```

```

58     } catch (SQLException e) {
59         throw new DataAccessException("Error fetching user", e);
60     }
61 }
62
63 @Override
64 public List<User> findAll() {
65     String sql = "SELECT * FROM users";
66     try (Connection conn = JDBCUtil.getConnection();
67         PreparedStatement ps = conn.prepareStatement(sql)) {
68         ResultSet rs = ps.executeQuery();
69         List<User> list = new ArrayList<>();
70         while (rs.next()) list.add(mapRow(rs));
71         return list;
72     } catch (SQLException e) {
73         throw new DataAccessException("Error fetching users", e);
74     }
75 }
76
77 @Override
78 public boolean delete(Integer id) {
79     String sql = "DELETE FROM users WHERE user_id=?";
80     try (Connection conn = JDBCUtil.getConnection();
81         PreparedStatement ps = conn.prepareStatement(sql)) {
82         ps.setInt(1, id);
83         return ps.executeUpdate() > 0;
84     } catch (SQLException e) {
85         throw new DataAccessException("Error deleting user", e);
86     }
87 }
88
89 private User mapRow(ResultSet rs) throws SQLException {
90     User u = new User();
91     u.setUserId(rs.getInt("user_id"));
92     u.setUsername(rs.getString("username"));
93     u.setPassword(rs.getString("password"));
94     u.setFullName(rs.getString("full_name"));
95     u.setEmail(rs.getString("email"));
96     u.setRole(rs.getString("role"));
97     return u;
98 }
99 }

```

Description:

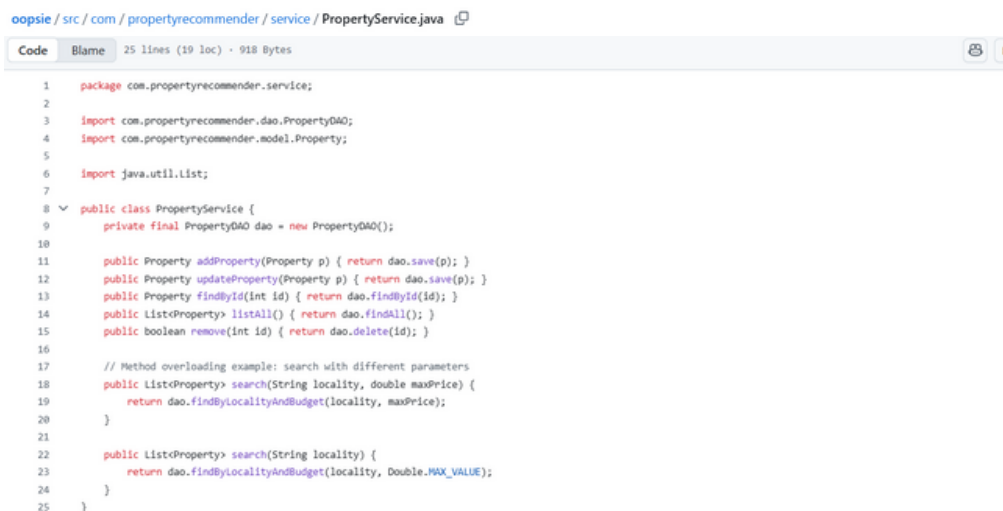
- The UserDAO module serves as the data access object responsible for all database operations related to user information. It provides methods to create, read, update, and delete (CRUD) user records in the database.
- This module manages connections to the backend database, handling SQL queries and transactions securely and efficiently. It abstracts the persistence logic away from the higher-level business logic in the system.
- UserDAO encapsulates the retrieval and storage of user credentials, profile details, and preferences. It also implements validation checks and error handling to ensure data integrity and consistency during database interactions.

7.2 Property Management Module

This class is part of the Property Management Module, responsible for handling business-level operations related to property data. It provides methods to add, update, retrieve, delete, and search properties, acting as a bridge between the controller layer and the data access layer. Method overloading is used to offer flexible property search options based on locality and budget.

Implementation Files (Github):

- `src/com/propertyrecommender/service/PropertyService.java`
- `src/com/propertyrecommender/model/Property.java`
- `src/com/propertyrecommender/dao/PropertyDAO.java`



```
oopsie / src / com / propertyrecommender / service / PropertyService.java
Code Blame 25 lines (19 loc) · 918 Bytes
1 package com.propertyrecommender.service;
2
3 import com.propertyrecommender.dao.PropertyDAO;
4 import com.propertyrecommender.model.Property;
5
6 import java.util.List;
7
8 public class PropertyService {
9     private final PropertyDAO dao = new PropertyDAO();
10
11     public Property addProperty(Property p) { return dao.save(p); }
12     public Property updateProperty(Property p) { return dao.save(p); }
13     public Property findById(int id) { return dao.findById(id); }
14     public List<Property> listAll() { return dao.findAll(); }
15     public boolean remove(int id) { return dao.delete(id); }
16
17     // Method overloading example: search with different parameters
18     public List<Property> search(String locality, double maxPrice) {
19         return dao.findByLocalityAndBudget(locality, maxPrice);
20     }
21
22     public List<Property> search(String locality) {
23         return dao.findByLocalityAndBudget(locality, Double.MAX_VALUE);
24     }
25 }
```

Description:

- The PropertyService module acts as an intermediary between the data access layer (DAO) and the presentation layer. It encapsulates core business logic related to property management, including validation, processing, and coordination of property-related operations.
- This service provides methods for adding, updating, deleting, and retrieving property details. It supports complex queries and filtering based on user inputs and business rules, ensuring the correct data is fetched and manipulated.
- PropertyService integrates with other system modules such as Recommendation and User Management to fetch relevant data and deliver comprehensive property information.

7.3 Recommendation Engine Module

This class is part of the Recommendation Module, providing a simple rule-based engine that suggests properties based on locality, budget, and BHK preference. It ranks available properties by assigning scores—rewarding BHK matches and lower prices—to generate an ordered list of best-fit recommendations.

Implementation Files (Github):

- `src/com/propertyrecommender/service/RecommendationService.java`

```
oopsie / src / com / propertyrecommender / service / RecommendationService.java  
1  package com.propertyrecommender.service;
2  import com.propertyrecommender.model.Property;
3  import java.util.*;
4  import java.util.stream.Collectors;
5  public class RecommendationService {
6      private final PropertyService propertyService;
7
8      public RecommendationService(PropertyService propertyService) {
9          this.propertyService = propertyService;
10     }
11     // Filter by locality and budget and then sort by price asc (simple ranking)
12     public List<Property> recommend(String locality, double maxPrice, int bhkPreference) {
13         List<Property> candidates = propertyService.search(locality, maxPrice);
14         // Score properties: match bhk gives extra score, lower price better rank.
15         Map<Property, Integer> score = new HashMap<>();
16         for (Property p : candidates) {
17             int s = 0;
18             if (p.getBhk() == bhkPreference) s += 10;
19             // price-based score (lower price higher score)
20             s += (int)(1000000 / (1 + p.getPrice()));
21             score.put(p, s);
22         }
23         return score.entrySet().stream()
24             .sorted(Map.Entry.<Property,Integer>comparingByValue().reversed())
25             .map(Map.Entry::getKey)
26             .collect(Collectors.toList());
27     }
28 }
29
```

Description:

- The RecommendationService module encapsulates the core algorithms and business logic used to generate personalized property suggestions. It analyzes user preferences, search histories, and available property data to deliver relevant recommendations.
- This service coordinates with the PropertyService to retrieve property details and the UserService to access user-specific data, combining these inputs to refine and tailor its output.
- It applies ranking algorithms to order recommendations by relevance and supports dynamic filtering to adapt suggestions based on changing user criteria or feedback.

7.4 Database Management Module

This class belongs to the Configuration Module, storing essential database connection details such as URL, username, and password. It centralizes all DB settings in one place, ensuring easier maintenance and cleaner code across the application.

Implementation Files (Github):

- `src/com/propertyrecommender/config/DBConfig.java`

```
oopsie / bin / com / propertyrecommender / config / DBConfig.class  ⓘ
1 package com.propertyrecommender.config;
2
3 public class DBConfig {
4     public static final String URL =
5         "jdbc:mysql://localhost:3306/property_recommender?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC";
6
7     public static final String USER = "root";
8     public static final String PASS = "YourNewPassword123!";
9 }
```

Description:

- DBConfig manages the configuration settings required to establish and maintain connections to the database. This includes loading database URLs, credentials, driver details, and connection properties.
- The module may implement or configure connection pooling to optimize resource usage, reduce connection latency, and handle multiple simultaneous database requests efficiently.
- DBConfig centralizes all database connectivity parameters, allowing other parts of the system to obtain database connections in a consistent, reusable manner, thereby promoting modularity and ease of maintenance.

7.5 Booking and Payment Module

This class is part of the Booking Management Module, handling end-to-end operations such as booking a property, cancelling a booking, and listing all bookings. It coordinates with both property and booking data layers to update availability and maintain consistent booking status.

Implementation Files (Github):

- `src/com/propertyrecommender/service/BookingService.java`
- `src/com/propertyrecommender/dao/BookingDAO.java`
- `src/com/propertyrecommender/model/Booking.java`

oopsie / src / com / propertyrecommender / service / BookingService.java 

Code Blame 43 lines (37 loc) · 1.54 KB

```
1 package com.propertyrecommender.service;
2
3 import com.propertyrecommender.dao.BookingDAO;
4 import com.propertyrecommender.dao.PropertyDAO;
5 import com.propertyrecommender.model.Booking;
6 import com.propertyrecommender.model.Property;
7
8 import java.sql.Timestamp;
9 import java.util.List;
10
11 public class BookingService {
12     private final BookingDAO bookingDAO = new BookingDAO();
13     private final PropertyDAO propertyDAO = new PropertyDAO();
14
15     public Booking bookProperty(int propertyId, int userId, double amount) {
16         Property p = propertyDAO.findById(propertyId);
17         if (p == null || !p.isAvailable()) throw new RuntimeException("Property not available");
18         Booking b = new Booking();
19         b.setPropertyId(propertyId);
20         b.setUserId(userId);
21         b.setBookingDate(new Timestamp(System.currentTimeMillis()));
22         b.setStatus("BOOKED");
23         b.setAmount(amount);
24         Booking saved = bookingDAO.save(b);
25         // set property unavailable
26         p.setAvailable(false);
27         propertyDAO.save(p);
28         return saved;
29     }
30
31     public boolean cancelBooking(int bookingId) {
32         Booking b = bookingDAO.findById(bookingId);
33         if (b == null || "CANCELLED".equals(b.getStatus())) return false;
34         b.setStatus("CANCELLED");
35         bookingDAO.save(b);
36         // make property available
37         Property p = propertyDAO.findById(b.getPropertyId());
38         if (p != null) { p.setAvailable(true); propertyDAO.save(p); }
39         return true;
40     }
41
42     public List<Booking> listAllBookings() { return bookingDAO.findAll(); }
43 }
```


CHAPTER 8 – RESULTS

1. User Menu

This is the main menu of the Property Recommender system, providing entry points for all user types. Users can log in with existing credentials, register for a new account, or proceed as a guest with limited access. The menu also offers a convenient option to exit the application directly.

```
=== Property Recommender ===  
1. Login  
2. Register  
3. Continue as Guest  
0. Exit
```

2. User Registration

This screenshot displays the registration process in the Property Recommender system. The user is prompted to input a username, password, full name, and email address, then select a role as either buyer or seller. Upon successful registration as a seller, the user receives a confirmation and is instructed to log in to proceed.

```
=== Property Recommender ===  
1. Login  
2. Register  
3. Continue as Guest  
0. Exit  
Choose: 2  
  
=== Register ===  
Choose username: Cristiano  
Password: Ron7  
Full name: Cristiano Ronaldo  
Email: ronaldo7@goat.com  
  
Select Role:  
1. Buyer  
2. Seller  
Choose: 2  
Registered successfully as SELLER. Please login.
```

3. User Login

This screenshot illustrates the login process in the Property Recommender system. The user selects the login option from the main menu and is prompted to enter a username and password. Successful authentication provides access to personalized features and services within the application.

```
=== Property Recommender ===
1. Login
2. Register
3. Continue as Guest
0. Exit
Choose: 1
Username: Cristiano
Password: Ron7
Welcome Cristiano Ronaldo
```

4. Seller Menu

This screenshot shows the Seller Menu in the Property Recommender system. The seller selects the "Add Property" option and enters details such as title, description, location, BHK, and price. After submitting the information, the system confirms that the property has been added successfully to their listings.

Add property

```
=== Seller Menu ===
1. Add Property
2. Update My Property
3. Delete My Property
4. View My Properties
5. Logout
Choose: 1
Title: Luxury 7 BHK Flats
Description: Gated Community
Locality: Thirmalgiri
BHK: 7
Price: 10000000
Property added successfully!
```

Update property

This screenshot shows the "Update Property" feature in the Seller Menu. The seller selects a property to update, modifies fields like title, description, location, BHK, and price, and submits the updated details. A confirmation message indicates that the property information has been successfully updated in the system.

```
=== Seller Menu ===
1. Add Property
2. Update My Property
3. Delete My Property
4. View My Properties
5. Logout
Choose: 2
Property[id=3, title=Luxury 7 BHK Flats, locality=Thirmalgiri, bhk=7, price=10000000.00, available=true]
Property ID to update: 3
Title (Luxury 7 BHK Flats): Ultra-Luxury 7 BHKs
Description: Gated Community
Locality: Thirumalpet
BHK: 7
Price: 10000000
Property updated successfully!
```

View my properties

This screenshot displays the "View My Properties" feature from the Seller Menu. Upon selecting this option, the seller can see a summary of their listed properties, including details like property ID, title, locality, BHK, price, and availability status. This allows sellers to keep track of and manage their active property listings easily.

```
=== Seller Menu ===
1. Add Property
2. Update My Property
3. Delete My Property
4. View My Properties
5. Logout
Choose: 4
Property[id=3, title=Ultra-Luxury 7 BHKs, locality=Thirumalpet, bhk=7, price=10000000.00, available=true]
```

Delete Property

This screenshot demonstrates the "Delete My Property" feature in the Seller Menu. The seller is shown a summary of their listed property and prompted to enter the property ID for deletion.

```
=== Seller Menu ===
1. Add Property
2. Update My Property
3. Delete My Property
4. View My Properties
5. Logout
Choose: 3
Property[id=3, title=Ultra-Luxury 7 BHKs, locality=Thirumalpet, bhk=7, price=10000000.00, available=true]
Property ID to delete: 3
Property deleted.
```

5. Buyer Menu

This screenshot displays the "Search / Recommend Properties" feature from the Buyer Menu. The user specifies preferences such as locality, maximum price, and preferred BHK (bedrooms). Based on the entered criteria, the system recommends matching properties, displaying details like property ID, title, locality, BHK, price, and availability status for easy comparison and selection.

Search property

```
=== Buyer Menu ===
1. Search / Recommend Properties
2. Book a Property
3. My Profile
4. Logout
Choose: 1
Preferred locality (partial match): Thirmal
Max price: 1000000000
Preferred BHK (0 to ignore): 0
Recommended Properties:
PropertyId=6, title=Ballondor, locality=Thirmalgiri, bhk=7, price=10000000.00, available=true]
```

Book Property

This screenshot shows the "Book a Property" feature from the Buyer Menu. The buyer selects the property to book by entering its ID and then provides the payment amount. The system processes the request and confirms that the property has been successfully booked.

```
=== Buyer Menu ===
1. Search / Recommend Properties
2. Book a Property
3. My Profile
4. Logout
Choose: 2
Property ID to book: 6
Enter payment amount: 10000000
Booked successfully!
```

My Profile

This screenshot shows the "My Profile" option from the Buyer Menu. Upon selection, the system displays the user's account details, including user ID, username, full name, and role. The user is then prompted to update their full name if desired, allowing for easy profile management.

```
=== Buyer Menu ===
1. Search / Recommend Properties
2. Book a Property
3. My Profile
4. Logout
Choose: 3
User[id=4, username=buyer1, name=, role=BUYER]
Change full name? (y/N): n
```

6. Admin Menu

This screenshot displays the Admin Dashboard in the Property Recommender system. The admin can choose to manage properties, manage users, view all bookings, or logout. Within the property management section, the admin is provided options to add, update, delete, or list all properties. The interface then lists detailed information about all available property records, enabling effective administrative oversight.

List all properties as admin

```
=== Admin Dashboard ===
1. Manage Properties
2. Manage Users
3. View All Bookings
4. Logout
Choose: 1

1. Add Property
2. Update Property
3. Delete Property
4. List All Properties
Choose: 4
Property[id=1, title=Metropolis, locality=Gachibowli, bhk=3, price=20000000.00, available=false]
Property[id=2, title=hahahah, locality=Gachibowli, bhk=3, price=100.00, available=false]
Property[id=4, title=Goat Villas, locality=Gachibowli, bhk=4, price=12345.00, available=true]
Property[id=5, title=Lake-View Apartments, locality=Kukatpally, bhk=3, price=67890.00, available=true]
Property[id=6, title=Ballondor, locality=Thirumalgar, bhk=7, price=10000000.00, available=false]
Property[id=7, title=Marvel, locality=Bachupally, bhk=5, price=222222.00, available=true]
```

Manage users

This screenshot shows the "Manage Users" feature in the Admin Dashboard of the Property Recommender system. The admin sees a list of all registered users with their IDs, usernames, full names, and roles (such as ADMIN, BUYER, or SELLER). The admin is prompted to enter a user ID to delete a user or enter 0 to skip, supporting efficient user management and system maintenance.

```
=== Admin Dashboard ===
1. Manage Properties
2. Manage Users
3. View All Bookings
4. Logout
Choose: 2
User[id=1, username=admin, name=System Admin, role=ADMIN]
User[id=2, username=Ronaldo7, name=Veerabhadra Yerram, role=BUYER]
User[id=3, username=Goat7, name=Goat, role=BUYER]
User[id=4, username=buyer1, name=, role=BUYER]
User[id=5, username=seller1, name=Ravi Seller, role=SELLER]
User[id=8, username=Sabrina, name=Sabrina Carpenter, role=BUYER]
User[id=9, username=Cristiano, name=Cristiano Ronaldo, role=SELLER]
Delete user ID (0 to skip): 0
```

View all booked properties

The image shows an admin dashboard menu for a property booking system, featuring options to manage properties, users, view bookings, and logout. It also displays a list of booking records with details like property ID, user ID, status as "BOOKED", and the booking amount.

```
=== Admin Dashboard ===
1. Manage Properties
2. Manage Users
3. View All Bookings
4. Logout
Choose: 3
Booking[id=1, propertyId=1, userId=4, status=BOOKED, amount=20000000.00]
Booking[id=2, propertyId=2, userId=4, status=BOOKED, amount=100.00]
Booking[id=3, propertyId=6, userId=4, status=BOOKED, amount=10000000.00]
```


4. Guest Menu

The image displays a console output for a "Property Recommender" system, providing options to log in, register, continue as guest, or exit. As a guest, several properties are listed with details like title, locality, BHK type, price, and availability status.

```
=== Property Recommender ===
1. Login
2. Register
3. Continue as Guest
0. Exit
Choose: 3
Browsing as guest...
Property[id=1, title=Metropolis, locality=Gachibowli, bhk=3, price=20000000.00, available=false]
Property[id=2, title=hahahah, locality=Gachibowli, bhk=3, price=100.00, available=false]
Property[id=4, title=Goat Villas, locality=Gachibowli, bhk=4, price=12345.00, available=true]
Property[id=5, title=Lake-View Apartments, locality=Kukatpally, bhk=3, price=67890.00, available=true]
Property[id=6, title=Ballondor, locality=Thirumalgar, bhk=7, price=10000000.00, available=false]
Property[id=7, title=Marvel, locality=Bachupally, bhk=5, price=222222.00, available=true]
```

CHAPTER 9 – CONCLUSIONS & FUTURE SCOPE

Conclusion

The Property Recommendation System (PRS) developed in this project successfully addresses key challenges in the real estate search and recommendation domain by leveraging object-oriented programming principles and Java technologies. The modular architecture ensures each component—from user management and property handling to recommendation logic and database interaction—is clearly defined, reusable, and maintainable. The system efficiently organizes property data, provides personalized suggestions, and delivers a smooth user interaction via a console-based interface.

Through the implementation, the project showcased the practical application of core Java concepts such as inheritance, polymorphism, encapsulation, interfaces, and database connectivity using JDBC. The system's menu-driven approach offers an intuitive experience for users of varying technical skills, while the use of collections and data abstraction guarantees efficient data handling.

The PRS is scalable and extensible, permitting easy addition of new property categories, recommendation algorithms, or interface enhancements. Security and reliability measures incorporated safeguard user data and ensure consistent performance under normal conditions.

Future Scope

Despite its strengths, the current PRS implementation leaves several avenues open for improvement and expansion:

- **Graphical User Interface (GUI):** Transitioning from a console-based interface to a rich web or mobile application would significantly improve user accessibility and engagement.
- **Advanced Recommendation Algorithms:** Incorporating machine learning techniques such as collaborative filtering, content-based filtering, or hybrid methods could produce smarter, more context-aware recommendations.
- **Real-time Data Integration:** Linking with live property databases and APIs will ensure up-to-date listings and pricing, tackling the common issue of stale data in current platforms.
- **Enhanced User Profiles and Feedback:** Adding comprehensive user profiles and feedback mechanisms can facilitate adaptive recommendations and improve user satisfaction.
- **Payment Gateway Integration:** Developing secure, multi-mode payment processing systems will support complete end-to-end booking management.

REFERENCES

1. Oracle Corporation. (2025). Java SE Documentation. Oracle.com
<https://docs.oracle.com/javase/>
2. Oracle Corporation. (2025). The Java Tutorials: Custom Networking. Oracle.com
<https://docs.oracle.com/javase/tutorial/networking/>
3. Oracle Corporation. (2025). JDBC API Documentation. Oracle.com
<https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc/>
4. Oracle Corporation. (2025). Java Platform, Standard Edition Security Developer's Guide. Oracle.com
5. Eclipse Foundation. (2025). Eclipse IDE Documentation. Eclipse.org
<https://www.eclipse.org/documentation/>
6. Oracle Corporation. (2025). MySQL 8.0 Reference Manual. MySQL.com
<https://dev.mysql.com/doc/refman/8.0/en/>
7. Oracle Corporation. (2025). MySQL Workbench Manual. MySQL.com
8. Schildt, Herbert. (2018). Java: The Complete Reference (11th Ed.). McGraw-Hill Education.
9. Sierra, Kathy & Bates, Bert. (2022). Head First Java (3rd Ed.). O'Reilly Media.
10. Liang, Y. Daniel. (2019). Introduction to Java Programming and Data Structures (11th Ed.). Pearson.
11. Oracle Corporation. (2025). Java Collections Framework Documentation. Oracle.com
<https://docs.oracle.com/javase/8/docs/technotes/guides/collections/>
12. Oracle Corporation. (2025). Java I/O Fundamentals. Oracle.com
<https://docs.oracle.com/javase/tutorial/essential/io/>
13. Oracle Corporation. (2025). Java Concurrency Tutorial. Oracle.com
<https://docs.oracle.com/javase/tutorial/essential/concurrency/>
14. Oracle Corporation. (2025). Java Annotations Tutorial. Oracle.com
<https://docs.oracle.com/javase/tutorial/java/annotations/>
15. Oracle Corporation. (2025). Java Networking and Proxies. Oracle.com
<https://docs.oracle.com/javase/8/docs/technotes/guides/net/proxies.html>
16. Baeldung. (2025). Guide to Java REST API Development. Baeldung.com
<https://www.baeldung.com/rest-with-spring-series>

- 17.Oracle Corporation. (2025). Java Security Architecture. Oracle.com
<https://docs.oracle.com/javase/8/docs/technotes/guides/security/>
- 18.Stack Overflow. (2019). Java JDBC and Database Connectivity Q&A.
<https://stackoverflow.com/questions/tagged/jdbc>
- 19.Oracle Corporation. (2025). Java Exception Handling Best Practices. Oracle.com
<https://docs.oracle.com/javase/tutorial/essential/exceptions/>
- 20.Oracle Blogs. (2025). Best Practices for Java Application Development.
Oracle.com/blogs
- 21.GeeksforGeeks - Java Programming Tutorials
<https://www.geeksforgeeks.org/java/>
- 22.JavaTpoint - Java Tutorials & Examples
<https://www.javatpoint.com/java-tutorial>
- 23.TutorialsPoint - Java Programming
<https://www.tutorialspoint.com/java/index.htm>
- 24.Oracle Java Documentation
<https://docs.oracle.com/en/java/>
- 25.W3Schools - Java Tutorial
<https://www.w3schools.com/java/>